

Sistema de Gestión Farmacéutica y Control de Medicamentos

FarmaGest — Trabajo Práctico Integrador · Entrega Final

Asignatura	Algoritmos y Estructuras de Datos II
Docente	Fernández Buttera, Alfonso Martín
Integrantes	Abosch, Nicolás · Hirschfeldt, Guido Andrés · Sznajderhaus, Federico Eiras, Ramiro Iván · Estrada, Axel Alexis · Tettamanti, Nicolás
Lenguaje	Java
Fecha de entrega	Junio de 2026

Índice

1. Introducción
2. Descripción del problema y alcance
3. Diseño estructural de la solución
4. Diseño funcional integrado
5. Definición de los Tipos de Datos Abstractos (TDA)
6. Selección y justificación de estructuras de datos
7. Diagrama estructural y funcional de TDA
8. Controlador o motor de resolución
9. Implementación parcial consolidada
10. Fragmentos de código relevantes
11. Escenario de simulación completo
12. Resultados obtenidos
13. Análisis de eficiencia
14. Alternativas evaluadas y decisiones descartadas
15. Limitaciones de la solución
16. Mejoras y evolución futura
17. Conclusiones

1. Introducción

El presente trabajo desarrolla **FarmaGest**, una solución algorítmica para la gestión de una cadena farmacéutica de mediana escala con múltiples sucursales. El sistema resuelve cinco problemas operativos centrales: el control de medicamentos por fecha de vencimiento bajo política FEFO (*First Expired, First Out*), la gestión de stock por sucursal, la generación automática de pedidos de reposición a proveedores, la búsqueda de sustitutos terapéuticos y el seguimiento individual de lotes.

El enfoque adoptado parte del modelado abstracto mediante Tipos de Datos Abstractos (TDA) y la selección estratégica de estructuras de datos que garanticen eficiencia en las operaciones más frecuentes. Este documento es autónomo: integra el análisis del problema, el diseño estructural y funcional, la definición final de los TDA, la implementación parcial en Java, un escenario de simulación con resultados reales de ejecución y el análisis de eficiencia, incorporando las correcciones señaladas en las entregas previas.

2. Descripción del problema y alcance

2.1 Contexto

Una cadena farmacéutica con sucursales distribuidas geográficamente debe despachar siempre el lote más próximo a vencer (FEFO), evitando pérdidas por vencimiento; determinar automáticamente qué lote despachar ante varios disponibles; gestionar el inventario de cada sucursal de forma eficiente; generar pedidos al proveedor cuando el stock cae por debajo de un umbral; resolver, ante un medicamento sin stock, cuál es el sustituto disponible más cercano en la red de equivalencias terapéuticas; y mantener trazable cada lote (número, vencimiento, cantidad y proveedor).

2.2 Alcance funcional

El sistema se organiza en cinco módulos funcionales, cada uno asociado a operaciones concretas:

Módulo	Responsabilidad	Operaciones principales
Inventario	Gestión de medicamentos y lotes por sucursal.	agregarLote, consultarStock, hayStock
Despacho	Selección automática del lote a dispensar según FEFO.	despacharMedicamento, proximoAVencer
Pedidos	Generación y gestión de órdenes de reposición.	generarPedido, procesarPedido
Sustitutos	Consulta y navegación de la red de equivalencias.	buscarSustituto, agregarSustitucion
Seguimiento de lotes	Trazabilidad de cada lote ingresado.	número, vencimiento, cantidad, proveedor por lote

2.3 Restricciones y supuestos

- Cada medicamento tiene un código único e irrepetible dentro del catálogo.
- Cada sucursal se identifica por un ID único. El sistema contempla hasta 50 sucursales y hasta 10.000 medicamentos distintos.
- La sustitución terapéutica es simétrica: si A es sustituto de B, B es sustituto de A.
- Un medicamento puede tener múltiples lotes simultáneos; cada lote es una unidad indivisible de trazabilidad.

2.4 Fuera de alcance

No se contempla interfaz gráfica, persistencia en base de datos, control de usuarios/permisos ni facturación. El foco está en el diseño estructural, la integración de TDA y el análisis de eficiencia, no en un producto comercial terminado.

3. Diseño estructural de la solución

La solución se organiza en cuatro capas de paquetes Java. El **modelo** contiene las entidades del dominio; las **estructuras** implementan los TDA; el **controlador** coordina y aplica las reglas de negocio; y un programa de prueba ejecuta la simulación integrada.

Componente	Responsabilidad	Se relaciona con
FarmaGestController	Único punto de acceso externo. Aplica las reglas de negocio (FEFO, umbral, sustituto) y coordina los TDA. Ningún TDA contiene lógica de dominio.	Los cuatro TDA principales
CatalogoMedicamentos	Catálogo global con acceso por código.	Consultado por el controlador antes de cada operación
Inventario	Stock por par (sucursal, medicamento) y umbrales.	Contiene una ColaDeLotes por par; consultado por RedDeSustitutos
ColaDeLotes	Cola con prioridad de lotes por vencimiento (FEFO) + acumulador de stock.	Administrada por el Inventario
ColaDePedidos	Cola FIFO de órdenes de reposición.	Alimentada por el controlador; procesada para reponer Inventario
RedDeSustitutos	Grafo no dirigido de equivalencias terapéuticas.	Consulta el Inventario durante el BFS

4. Diseño funcional integrado

Esta sección describe FarmaGest como un sistema completo: qué entradas recibe, qué decisiones toma el controlador, qué estructuras consulta o modifica y qué resultados produce. El circuito principal es continuo y enlaza las seis operaciones del sistema.

Operación (entrada)	Decisión del controlador	Estructuras consultadas / modificadas	Resultado (salida)
Ingreso de lote (suc, med, lote)	Valida que el medicamento exista.	Consulta Catálogo; modifica ColaDeLotes del Inventario.	Lote insertado; stock actualizado.
Despacho (suc, med, cantidad)	Verifica stock y aplica FEFO; si el stock queda $\leq$ umbral, ordena reposición.	Consulta Catálogo e Inventario; modifica ColaDeLotes y, eventualmente, ColaDePedidos.	Unidades despachadas; pedido automático si corresponde.
Procesar pedido	Toma el pedido más antiguo (FIFO).	Modifica ColaDePedidos (extrae) e Inventario (reponer lote).	Stock repuesto.
Buscar sustituto (suc, med)	Si el medicamento tiene stock, no busca; si no, recorre la red.	Consulta Inventario y RedDeSustitutos (BFS).	Sustituto disponible más cercano, o ninguno.
Purga de vencidos	Operación batch periódica.	Recorre el Inventario; modifica cada ColaDeLotes.	Lotes vencidos eliminados; stock corregido.

El recorrido funcional principal —ilustrado en la Sección 7— encadena: *ingreso de lote* → *validación en catálogo* → *consulta de stock* → *despacho FEFO* → *detección de umbral bajo* → *generación de pedido* → *procesamiento de reposición* → *búsqueda de sustituto* → *purga de vencidos*. De este modo el sistema se lee como una solución continua y no como operaciones aisladas.

5. Definición de los Tipos de Datos Abstractos (TDA)

Cada TDA define un **contrato conceptual** —responsabilidades, operaciones y precondiciones— con independencia de su representación concreta. La columna «Estructura Java» indica la implementación elegida, manteniendo separados el contrato del TDA y la estructura que lo realiza.

TDA	Responsabilidad y operaciones principales	Estructura Java (tipo)
Medicamento	Identidad y metadatos del producto (código, nombre, droga, presentación). No contiene stock. getCodigo, equals, hashCode. Precondición: código único no nulo.	POJO inmutable
Lote	Unidad mínima de trazabilidad. restarUnidades, estaVencido, compareTo. Precondición: cantidad > 0; orden natural por vencimiento.	Clase con Comparable<Lote>
CatalogoMedicamentos	Catálogo con acceso por código. registrar, buscarPorCodigo, contiene. Precondición: código no nulo.	Diccionario — HashMap (acceso directo)
Inventario	Stock por (sucursal, medicamento) y umbrales. agregarLote, consultarStock, hayStock, umbralBajo.	Diccionario múltiple — HashMap anidado (acceso directo)
ColaDeLotes	Mantiene los lotes ordenados por vencimiento (FEFO). agregar, proximoAVencer, despachar, hayStock, eliminarVencidos. Precondición de despacho: hayStock.	Cola con Prioridad — PriorityQueue (jerárquica)
ColaDePedidos	Pedidos de reposición en orden de llegada. encolarPedido, procesarPedido, verProximoPedido (FIFO).	Cola — LinkedList como Queue (lineal)
RedDeSustitutos	Equivalencias terapéuticas. agregarNodo, agregarArista, buscarSustitutoDisponible (BFS). Relación simétrica.	Grafo no dirigido — lista de adyacencia (relacional)

6. Selección y justificación de estructuras de datos

Cada estructura corresponde a un TDA o tema del programa de la asignatura (Unidades 1 a 7) y se realiza en Java sobre el Java Collections Framework, según habilita el enunciado.

Estructura (Java)	TDA / tema de clase	Unidad	Justificación técnica
LinkedList como Queue (FIFO)	TDA Cola	Unidad 2	Los pedidos se atienden en orden de generación, sin urgencia diferenciada: semántica FIFO. Encolar/desencolar en O(1).
PriorityQueue<Lote>	TDA Cola con Prioridad	Unidad 2	La política FEFO requiere recuperar siempre el lote de menor fecha de vencimiento: acceso al mínimo O(1), inserción/extracción O(log n).
HashMap<String, Medicamento>	TDA Diccionario (simple)	Unidad 3	La búsqueda por código es la operación más frecuente; acceso O(1) amortizado. Una estructura O(n) penalizaría cada operación.
HashMap anidado	TDA Diccionario (múltiple)	Unidad 3	Acceso O(1) al par (sucursal, medicamento) con dos consultas; más claro que una clave compuesta de tipo cadena.
HashSet de visitados	TDA Conjunto	Unidad 2	Evita reprocesar nodos y previene recorridos infinitos en el BFS sobre un grafo con ciclos.
Grafo + BFS (lista de adyacencia)	Grafo / recorrido en anchura	Unidad 7	La sustitución es simétrica y con ciclos: un grafo no dirigido la modela fielmente. Lista de adyacencia: O(V+A) en BFS y O(1) en vecinos; la matriz sería O(V ²) en espacio.

Quedan cubiertas las cuatro categorías exigidas: **lineal** (Cola FIFO), **acceso directo** (Diccionario simple y múltiple), **jerárquica** (Cola con Prioridad) y **relacional** (Grafo).

Sobre la Cola con Prioridad y el heap. El concepto aplicado es el TDA Cola con Prioridad (Unidad 2). La clase PriorityQueue de Java lo realiza internamente mediante un *heap* binario, lo que provee el costo O(log n). Como estructura vista en clase para el mismo fin, una cola con prioridad ordenada por fecha podría sostenerse sobre un árbol balanceado **AVL** (Unidad 6), que también garantiza O(log n).

7. Diagrama estructural y funcional de TDA

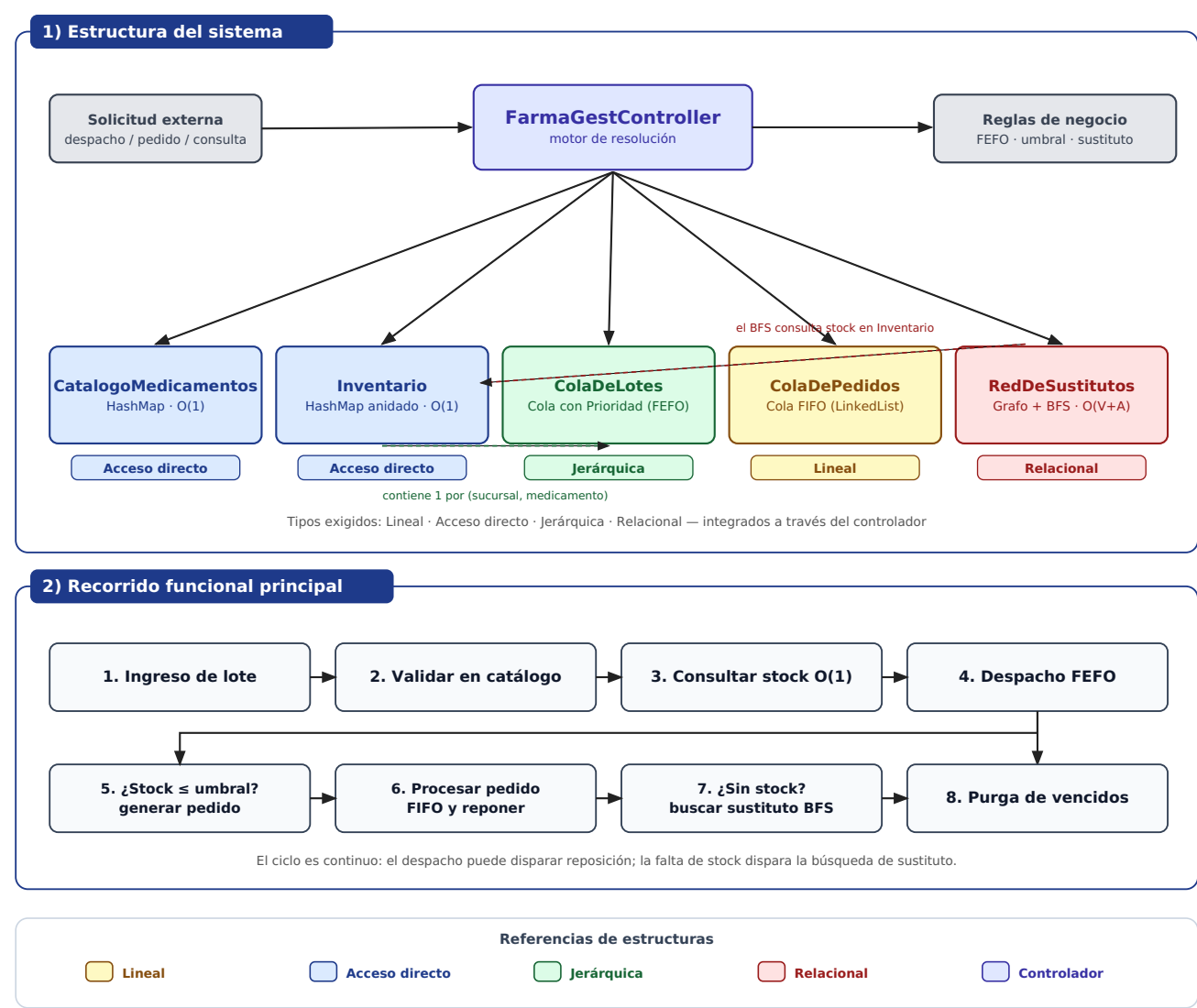


Figura 1 — Estructura del sistema (controlador + TDA por categoría) y recorrido funcional principal de FarmaGest.

El panel superior muestra el **FarmaGestController** como nodo central que recibe las solicitudes externas y delega en los TDA, agrupados por categoría estructural. Las flechas punteadas indican dos relaciones internas clave: el Inventario contiene una ColaDeLotes por cada par (sucursal, medicamento), y la RedDeSustitutos consulta el stock del Inventario en cada nodo visitado durante el BFS. El panel inferior muestra el recorrido funcional principal como un ciclo continuo.

8. Controlador o motor de resolución

El **FarmaGestController** es el único punto de acceso externo. Mantiene una instancia de cada uno de los cuatro TDA principales y concentra las reglas de negocio, de modo que los TDA permanecen como estructuras puras. La siguiente tabla resume, por método, qué estructuras invoca y qué regla aplica.

Método del controlador	Estructuras que invoca	Regla de negocio
despacharMedicamento	Catálogo (validar), Inventario (hayStock, consultarStock, despachar), ColaDePedidos (encolar)	FEFO + reposición automática si el stock queda $\leq$ umbral.
buscarSustituto	Catálogo, Inventario (hayStock), RedDeSustitutos (BFS)	Si hay stock no se busca; si no, se devuelve el sustituto disponible más cercano (excluyendo el origen).
procesarPedido	ColaDePedidos (poll), Inventario	Atiende el pedido más antiguo (FIFO) y repone el

	(agregarLote)	stock.
agregarLote	Catálogo (validar), Inventario (insertar en ColaDeLotes)	Solo se ingresan lotes de medicamentos registrados.
purgarVencidos	Inventario (recorrer), ColaDeLotes (eliminarVencidos)	Elimina lotes vencidos de todas las sucursales (batch).

9. Implementación parcial consolidada

Se implementaron en Java los seis TDA y el controlador, respetando los contratos definidos. El proyecto se organiza así:

farmagest/	
modelo/	
Medicamento.java	TDA Medicamento (POJO inmutable)
Lote.java	TDA Lote (Comparable<Lote>, cantidad mutable)
Pedido.java	Orden de reposición
estructuras/	
CatalogoMedicamentos.java	Diccionario simple (HashMap)
Inventario.java	Diccionario múltiple (HashMap anidado) + umbrales
ColaDeLotes.java	Cola con Prioridad (PriorityQueue) + acumulador stockTotal
ColaDePedidos.java	Cola FIFO (LinkedList como Queue)
RedDeSustitutos.java	Grafo no dirigido (lista de adyacencia) + BFS
controlador/	
FarmaGestController.java	Motor central (único punto de acceso)
EscenarioPrueba.java	main() con la simulación integrada

El código compila y ejecuta la simulación completa sin errores. Cada TDA expone únicamente las operaciones de su interfaz pública y oculta su representación interna.

10. Fragmentos de código relevantes

10.1 Lote — orden natural FEFO

El contrato que habilita la cola con prioridad: los lotes se ordenan por fecha de vencimiento ascendente, de modo que el de vencimiento más próximo queda siempre accesible.

```
public int compareTo(Lote otroLote) {
    return this.fechaVencimiento.compareTo(otroLote.fechaVencimiento);
}
```

10.2 ColaDeLotes — despacho FEFO con acumulador

Operación más crítica. Consume el lote de menor vencimiento; si se agota, lo extrae y continúa con el siguiente. El acumulador `stockTotal` se descuenta al final, en $O(1)$.

```
public void despachar(int cantidad) {
    if (cantidad <= 0) throw new IllegalArgumentException("La cantidad a despachar debe ser mayor a 0.");
    if (!hayStock()) throw new IllegalStateException("No hay stock disponible para despachar.");
    if (cantidad > stockTotal) throw new IllegalStateException(
        "Stock insuficiente: solicitado=" + cantidad + ", disponible=" + stockTotal);

    int restante = cantidad;
    while (restante > 0) {
        Lote actual = heap.peek();
        if (actual == null) break;

        if (actual.getCantidad() <= restante) {
            // Lote agotado completamente: extraer del heap - O(log n)
            restante -= actual.getCantidad();
            heap.poll();
        } else {
            // Lote con stock suficiente: descontar unidades - O(1)
            actual.restarUnidades(restante);
            restante = 0;
        }
    }
    stockTotal -= cantidad;
}
```

10.3 Inventario — ingreso de lote (diccionario múltiple)

Si es el primer lote de un par (sucursal, medicamento), se crea su `ColaDeLotes` consultando antes su existencia, al estilo del diccionario simple visto en clase.

```
public void agregarLote(String sucId, String medId, Lote lote) {
    validarSucursal(sucId);
    if (medId == null) throw new IllegalArgumentException("El ID de medicamento no puede ser nulo.");
    if (lote == null) throw new IllegalArgumentException("El lote no puede ser nulo.");
    Map<String, ColaDeLotes> medsDeSucursal = stock.get(sucId);
    if (!medsDeSucursal.containsKey(medId)) {
        medsDeSucursal.put(medId, new ColaDeLotes());
    }
    medsDeSucursal.get(medId).agregar(lote);
}
```

10.4 RedDeSustitutos — BFS de sustituto disponible

Recorrido en anchura con cola y conjunto de visitados. El nodo de origen se excluye explícitamente: se busca un sustituto *distinto* al medicamento solicitado.

```
public String buscarSustitutoDisponible(String medId, Inventario inventario, String sucId) {
    validarNodo(medId);
    if (inventario == null) throw new IllegalArgumentException("El inventario no puede ser nulo.");

    Set<String> visitados = new HashSet<>();
    Queue<String> cola = new LinkedList<>();

    visitados.add(medId);
    cola.offer(medId);

    while (!cola.isEmpty()) {
        String actual = cola.poll();

        // Excluir nodo origen: solo retornar sustitutos DISTINTOS al medicamento solicitado
        if (!actual.equals(medId) && inventario.hayStock(sucId, actual)) {
            return actual;
        }

        List<String> vecinos = adyacencia.get(actual);
        if (vecinos != null) {
            for (String vecino : vecinos) {
                if (!visitados.contains(vecino)) {

```

```

        visitados.add(vecino);
        cola.offer(vecino);
    }
}
}
return null;
}

```

10.5 FarmaGestController — despacho integrado

Integra Catálogo, Inventario y ColaDePedidos en un único flujo: valida, verifica stock con el acumulador, despacha por FEFO y dispara la reposición si corresponde.

```

public boolean despacharMedicamento(String sucId, String medId, int cantidad) {
    if (!catalogo.contiene(medId)) {
        throw new IllegalArgumentException("Medicamento no registrado en el catálogo: " + medId);
    }
    if (!inventario.hayStock(sucId, medId)) {
        System.out.println("[ADVERTENCIA] Sin stock para " + medId + " en " + sucId);
        return false;
    }
    if (inventario.consultarStock(sucId, medId) < cantidad) {
        System.out.println("[ADVERTENCIA] Stock insuficiente para " + medId + " en " + sucId);
        return false;
    }

    inventario.despachar(sucId, medId, cantidad);

    if (inventario.umbraBajo(sucId, medId)) {
        generarPedidoAutomatico(sucId, medId);
    }

    return true;
}

```

11. Escenario de simulación completo

La simulación reproduce la operación de una cadena con dos sucursales (SUC-PALERMO y SUC-BELGRANO), cinco medicamentos y una red de sustitutos, ejecutando en secuencia los seis flujos del sistema.

11.1 Datos de entrada

Medicamento	Nombre	Lote	Cant.	Vence
IBU-400	Ibuprofeno 400mg	L001	50	2025-06-30
IBU-400	Ibuprofeno 400mg	L002	30	2026-03-15
IBU-400	Ibuprofeno 400mg	L003	20	2026-12-01
AMOX-500	Amoxicilina 500mg	L010	8	2025-09-01
DICLOF	Diclofenac 50mg	L020	40	2026-06-01
AMOX-750	Amoxicilina 750mg	L030	25	2026-05-01

Red de sustitutos: IBU-400 ↔ IBU-600 ↔ DICLOF | AMOX-500 ↔ AMOX-750.

Umbrales (SUC-PALERMO): IBU-400 = 10 unidades | AMOX-500 = 5 unidades.

11.2 Flujos ejecutados

- Ingreso de los lotes (IBU-400 acumula 100 unidades).
- Despacho FEFO de IBU-400 (se consume primero L001, el de vencimiento más próximo).
- Despacho que deja AMOX-500 en umbral: se genera un pedido automático.
- Agotamiento de IBU-400 y AMOX-500 y búsqueda de sus sustitutos (BFS).
- Procesamiento del primer pedido pendiente (FIFO) y reposición de stock.
- Purga de lotes vencidos.

12. Resultados obtenidos

Salida real de consola obtenida al ejecutar EscenarioPrueba:

```
=== FARMAGEST – ESCENARIO DE PRUEBA ===

[SETUP] Sucursales registradas: SUC-PALERMO, SUC-BELGRANO
[SETUP] Medicamentos registrados: IBU-400, IBU-600, DICLOF, AMOX-500, AMOX-750
[SETUP] Red de sustitutos configurada.

--- PRUEBA 1: Ingreso de lotes ---
Stock IBU-400 en SUC-PALERMO: 100 u.
Stock AMOX-500 en SUC-PALERMO: 8 u.

--- PRUEBA 2: Despacho FEFO ---
Próximo lote a vencer (ANTES): Lote[nro=L001, vence=2025-06-30, cantidad=50, proveedor=ProveedorA]
Despacho 40u IBU-400: EXITOSO
Stock IBU-400 tras despacho: 60 u.
Próximo lote (DESPUÉS del despacho): Lote[nro=L001, vence=2025-06-30, cantidad=10, proveedor=ProveedorA]
Próximo lote tras agotar L001: Lote[nro=L002, vence=2026-03-15, cantidad=30, proveedor=ProveedorB]

--- PRUEBA 3: Pedido automático por umbral ---
Pedidos pendientes antes: 0
[PEDIDO GENERADO] Pedido[id=PED-1, sucursal=SUC-PALERMO, medicamento=AMOX-500, cantidad=100, proveedor=PROVEEDOR_DEFAULT]
Stock AMOX-500 tras despacho: 4 u.
Pedidos pendientes después: 1

--- PRUEBA 4: Búsqueda de sustituto disponible ---
[PEDIDO GENERADO] Pedido[id=PED-2, sucursal=SUC-PALERMO, medicamento=IBU-400, cantidad=100, proveedor=PROVEEDOR_DEFAULT]
Stock IBU-400 agotado: 0
Sustituto IBU-400: Medicamento[codigo=DICLOF, nombre=Diclofenac 50mg, droga=Diclofenac, presentacion=Comprimidos x 20]
[PEDIDO GENERADO] Pedido[id=PED-3, sucursal=SUC-PALERMO, medicamento=AMOX-500, cantidad=100, proveedor=PROVEEDOR_DEFAULT]
Stock AMOX-500 agotado: 0
Sustituto AMOX-500: Medicamento[codigo=AMOX-750, nombre=Amoxicilina 750mg, droga=Amoxicilina, presentacion=Comprimidos x 12]

--- PRUEBA 5: Procesamiento de pedido ---
Pedidos en cola: 3
  Pedido[id=PED-1, sucursal=SUC-PALERMO, medicamento=AMOX-500, cantidad=100, proveedor=PROVEEDOR_DEFAULT]
  Pedido[id=PED-2, sucursal=SUC-PALERMO, medicamento=IBU-400, cantidad=100, proveedor=PROVEEDOR_DEFAULT]
  Pedido[id=PED-3, sucursal=SUC-PALERMO, medicamento=AMOX-500, cantidad=100, proveedor=PROVEEDOR_DEFAULT]
[PEDIDO PROCESADO] Pedido[id=PED-1, sucursal=SUC-PALERMO, medicamento=AMOX-500, cantidad=100, proveedor=PROVEEDOR_DEFAULT] ->
lote ingresado: L-REPO-001
Stock AMOX-500 tras procesar pedido: 100 u.

--- PRUEBA 6: Purga de lotes vencidos ---
Stock DICLOF antes de purga: 40 u.
```

```
Stock AMOX-750 antes de purga: 25 u.  
[BATCH] Purga de lotes vencidos completada.  
Stock DICLOF después de purga: 0 u.  
Stock AMOX-750 después de purga: 0 u.
```

```
=== FIN DEL ESCENARIO DE PRUEBA ===
```

Corrección verificada. La búsqueda de sustituto para AMOX-500 devuelve ahora AMOX-750 —y no el propio AMOX-500— resolviendo la observación de la Entrega 2. El recorrido para IBU-400 atraviesa IBU-600 (sin stock) y retorna DICLOF, mostrando un sustituto a dos saltos. Al agotar IBU-400 y AMOX-500 por debajo de su umbral, el sistema genera los pedidos automáticos correspondientes (quedan tres en cola), y la purga elimina los lotes vencidos de DICLOF y AMOX-750.

13. Análisis de eficiencia

Operación	Complejidad	Estructura y justificación
buscarMedicamento	O(1)	Diccionario (HashMap): acceso por clave hash.
consultarStock / hayStock	O(1)	Acumulador stockTotal: lee un entero, no recorre los lotes.
proximoAVencer	O(1)	Acceso al mínimo de la cola con prioridad.
agregarLote	O(log n)	Inserción en la cola con prioridad.
despacharMedicamento	O(k log n)	k = lotes agotados durante el despacho; con despacho parcial de un lote, O(log n).
encolarPedido	O(1)	Inserción al final de la cola FIFO.
procesarPedido	O(log n)	Extracción FIFO O(1) + inserción de lote O(log n).
buscarSustitutoDisponible	O(V + A)	BFS sobre el grafo; consulta de stock O(1) por nodo.
eliminarVencidos	O(n log n)	Reconstrucción de la cola con prioridad.

13.1 Operaciones críticas

Las de mayor frecuencia determinan el rendimiento observable: **despacharMedicamento** (O(k log n) gracias a la cola con prioridad y el acumulador) y **buscarMedicamento** (O(1) por el diccionario).

13.2 Decisión clave: el acumulador stockTotal

Mantener un entero con el stock total de cada ColaDeLotes convierte la verificación de disponibilidad de O(n) —recorrer y sumar todos los lotes— a O(1). Su costo es mínimo: un incremento al agregar y un decremento al despachar o purgar, todos O(1). Sin él, el flujo de despacho sería O(n + log n) en lugar de O(k log n).

14. Alternativas evaluadas y decisiones descartadas

Decisión	Alternativa descartada	Estructura elegida	Motivo técnico
Gestión de lotes (FEFO)	Lista enlazada ordenada por vencimiento	Cola con Prioridad (PriorityQueue)	La lista ordenada requiere O(n) para insertar en la posición correcta; la cola con prioridad inserta en O(log n) y accede al mínimo en O(1).
Gestión de lotes (alt. 2)	ArrayList + ordenamiento periódico	Cola con Prioridad	Reordenar tras cada inserción es O(n log n) y opera en lote, ineficiente ante ingresos continuos.
Catálogo	TreeMap (árbol balanceado)	HashMap	El orden lexicográfico no es requerimiento; para búsqueda pura por clave, O(1) amortizado supera a O(log n).
Red de sustitutos	Árbol de sustitutos	Grafo no dirigido (lista de adyacencia)	La sustitución es simétrica y con ciclos; un árbol es acíclico y jerárquico, no captura la relación.
Representación del inventario	HashMap con clave compuesta "suc_med"	HashMap anidado	La clave compuesta es frágil (formato/escapado) y dificulta iterar por sucursal; el anidado es más claro e igual de eficiente.
Cola de pedidos	Cola con prioridad por urgencia	Cola FIFO	El enunciado no define urgencia diferenciada; la equidad FIFO modela el comportamiento real. La migración futura es directa porque el TDA aísla la implementación.

15. Limitaciones de la solución

- No hay persistencia: el estado se mantiene en memoria y se pierde al finalizar la ejecución.
- La purga de vencidos es O(n log n) por par y se ejecuta en lote; con gran volumen conviene agendarla fuera de los picos de operación.

- El sistema es de un solo hilo: no contempla concurrencia entre sucursales.
- El umbral de reposición y la cantidad a pedir son fijos por configuración; no consideran demanda histórica.
- El escenario utiliza fechas de prueba; en producción debería validarse que no se despachen lotes ya vencidos antes de aplicar FEFO.

16. Mejoras y evolución futura

- Incorporar persistencia (archivo o base de datos) para conservar el estado entre ejecuciones.
- Migrar la ColaDePedidos a una cola con prioridad por urgencia si se definen niveles de criticidad.
- Disparar la purga de vencidos a partir de un umbral de proximidad de vencimiento, no solo por lote vencido.
- Definir interfaces explícitas por TDA para formalizar la separación contrato/implementación.
- Ampliar la simulación a mayor volumen de sucursales, medicamentos y lotes para validar la escalabilidad.
- Agregar pruebas unitarias por TDA y métricas de cobertura.

17. Conclusiones

FarmaGest demuestra cómo la combinación de múltiples estructuras de datos, integradas a través de un controlador central, resuelve un problema realista de gestión farmacéutica. La cola con prioridad sostiene la política FEFO; el diccionario simple y el múltiple dan acceso $O(1)$ al catálogo y al stock; la cola FIFO ordena los pedidos; y el grafo con BFS resuelve la búsqueda de sustitutos. El acumulador de stock resultó la decisión de eficiencia más importante, llevando el flujo de despacho a $O(k \log n)$.

El trabajo evidencia que el valor de la solución no está en la cantidad de código, sino en el diseño estructural, la correcta integración de los TDA y la fundamentación de cada decisión en términos de eficiencia. Las estructuras seleccionadas se corresponden con los contenidos de la asignatura, y la simulación ejecutada confirma que el comportamiento real coincide con el diseño propuesto.